Discussion

The Zero-Mock Tradeoff I

The Zero-Mock testing policy is `template/`'s most distinctive design decision. By prohibiting all mock objects, we gain confidence that tests exercise real code paths—a pytest run against the template genuinely invokes `pandoc`, writes to disk, and parses real YAML. The cost is test duration: the full infrastructure test suite (~9,557 tests) runs for minutes rather than the sub-second execution typical of heavily-mocked suites. This manuscript deliberately reports no wall-clock figure, consistent with the Results section's discipline of declining timing claims without a versioned benchmark artifact.

The Zero-Mock Tradeoff II

We argue this tradeoff is strongly favorable for research software. Unlike web applications where millisecond latency and thousands of daily deploys demand fast feedback loops, research pipelines run infrequently (once per manuscript revision) and correctness vastly outweighs speed. A mocked test that passes while the real renderer fails is worse than a slow test that catches the failure. The analogy to statistical methodology is precise: just as Simmons et al.'s *researcher degrees of freedom* [@simmons2011falsepositive] inflate false-positive rates through undisclosed analytical flexibility, mock objects create *testing degrees of freedom* that make integration failures invisible. The Zero-Mock policy closes this loophole by the same mechanism that pre-registration [@nosek2018preregistration] closes the p-hacking loophole: removing flexibility before the fact. As Peng [@peng2011reproducible] argues, computational reproducibility requires independent verification—and mock-only tests verify assumptions rather than results. Garijo et al.'s FAIRsoft evaluator [@garijo2024fairsoft] identifies *executability* as a primary quality indicator; the Zero-Mock policy operationalizes executability at the unit level.